Binary Heaps and Priority Queues: Array Representations, Dynamic Operations, Greedy Optimization, and an O(N) In-Place Construction Proof

Abstract

This paper presents a formal, unified algorithmic analysis of binary heaps and priority queue data structures derived from classroom instruction on May 29, 2026. We trace the theoretical progression from dynamic greedy optimization (the Connect Ropes problem) to Complete Binary Tree (CBT) array representations. We formalize implicit array node indexing formulas, up-heapify (percolate-up) insertion, and down-heapify (percolate-down) min extraction operations, proving their O(log N) worst-case time bounds. A major contribution of this paper is a rigorous mathematical proof of Floyd's O(N) in-place buildHeap algorithm using infinite arithmetic-geometric series differentiation. A complete, compilable reference implementation of a generic MinHeap in Java is provided, along with a comparative analysis of priority queue structures and applications in kernel scheduling, graph shortest paths, and stream processing.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis

1 Introduction

Priority queues and binary heaps constitute fundamental structural building blocks in computer science, serving as essential components across dynamic greedy optimization, graph theory algorithms (e.g., Dijkstra's shortest path, Prim's minimum spanning tree), operating system kernel process scheduling, and event-driven simulations. In modern technical assessment frameworks, heap-based data structures represent approximately 36% of core data structures and algorithms (DSA) questions due to their unique balance of structural simplicity and mathematical elegance.

This paper presents a formal, unified algorithmic analysis of binary heaps and priority queues derived from classroom instruction on May 29, 2026. We investigate the theoretical foundations of min-heaps and max-heaps, beginning with the dynamic minimum extraction problem (Connect Ropes). We establish why simple sorted arrays or unindexed linear structures fail to maintain dynamic minimum bounds efficiently. Furthermore, we formalize array-based Complete Binary Tree (CBT) indexing schemes, analyze up-heapify (percolate-up) insertion and down-heapify (percolate-down) deletion operations, and present a complete, production-grade reference implementation in Java.

A primary highlight of this paper is a rigorous mathematical proof of Floyd's $O(N)$ in-place buildHeap algorithm. Using infinite arithmetic-geometric series differentiation, we prove that bottom-up heapification achieves $O(N)$ linear time complexity, improving upon naive $O(N \log N)$ repeated insertion strategies.

1.1 Preliminaries and Notation

Throughout this paper, we adopt the following standard notation:

- N: The total number of elements in the heap dataset.
- $H = \lfloor \log_2 N \rfloor$: The height of a Complete Binary Tree containing N nodes.
- $A[0 \dots N-1]$: A zero-indexed array storing complete binary tree node values.
- $\text{parent}(i) = \lfloor \frac{i-1}{2} \rfloor$: The array index of the parent of node i.
- $\text{left}(i) = 2i + 1$: The array index of the left child of node i.
- $\text{right}(i) = 2i + 2$: The array index of the right child of node i.

2 Dynamic Minimum Optimization: The Connect Ropes Problem

To illustrate the necessity of heap structures, we first analyze the classic greedy optimization problem: Minimum Cost to Connect Ropes.

2.1 Formal Problem Definition

Given an array $R = [r_0, r_1, \ldots, r_{N-1}]$ representing the lengths of N initial ropes, connect all ropes into a single unified rope. In a single operation, any two ropes of lengths r_i and r_j can be connected at a cost equal to their sum $r_i + r_j$, producing a new rope of length $r_i + r_j$. The goal is to determine the sequence of connections that minimizes the total accumulated cost.

2.2 Mathematical Formalization of the Greedy Choice

Let C_{total} represent the total cost of connecting N ropes. Each initial rope length r_k contributes to the total cost weighted by the number of merge operations d_k it undergoes:

$$C_{\text{total}} = \sum_{k=0}^{N-1} r_k \cdot d_k$$

where d_k represents the depth/frequency of rope r_k in the execution merge tree. To minimize C_{total}, smaller rope lengths must undergo higher merge frequencies d_k, while larger rope lengths must undergo minimal merge frequencies.

Theorem 1 (Greedy Choice Property for Minimum Cost Ropes). An optimal sequence of rope connections always selects the two current shortest available ropes at every step.

Proof. Let a and b be the two smallest elements in the active set of ropes. Suppose an optimal solution S^* merges a non-minimal element $c > a$ with b at step 1, leaving a to be merged at a later step with depth $d_a < d_c$. Exchanging the positions of a and c in the merge tree alters the total cost by:

$$\Delta C = (a \cdot d_c + c \cdot d_a) - (a \cdot d_a + c \cdot d_c) = (a - c)(d_c - d_a)$$

Since $a < c$ $(a - c < 0)$ and $d_c > d_a$ $(d_c - d_a > 0)$, we have $\Delta C < 0$, which contradicts the assumption that S^* was optimal. Thus, choosing the two smallest ropes at every step preserves optimality. □

2.3 Algorithmic Evaluation Across Data Paradigms

Table 1 evaluates different algorithmic paradigms for solving the Connect Ropes problem across $N - 1$ iterations.

TABLE 1
Algorithmic Paradigms for the Connect Ropes Problem

Paradigm / Approach	Min Selection per Step	Re-Insertion Cost	Total Time Complexity
Repeated Array Re-Sorting	$O(1)$	$O(N \log N)$	$O(N^2 \log N)$
Linear Min-Scan per Step	$O(N)$	$O(1)$	$O(N^2)$
Sorted List Shift Insertion	$O(1)$	$O(N)$ shift	$O(N^2)$
Min-Heap (Priority Queue)	$O(\log N)$	$O(\log N)$	$O(N \log N)$

```
long minCostToConnectRopes(int[] ropes) {
    PriorityQueue<Long> pq = new PriorityQueue<>();
    for (int r : ropes) pq.offer((long) r);

    long totalCost = 0;
    while (pq.size() > 1) {
        long min1 = pq.poll(); // O(log N)
        long min2 = pq.poll(); // O(log N)
        long cost = min1 + min2;
        totalCost += cost;
        pq.offer(cost);       // O(log N)
    }
    return totalCost;
}
```

3 Binary Heap Data Structure & Array Representation

A Binary Heap is an array object that can be viewed as a nearly Complete Binary Tree (CBT).

3.1 Structural Invariants

Definition 2 (Complete Binary Tree). A binary tree T of depth h is complete if all levels $0, 1, \ldots, h-1$ are completely filled, and level h is filled strictly from left to right without structural gaps.

Definition 3 (Heap Structural Invariants).
1) Min-Heap Invariant: For every node i other than root, $A[\text{parent}(i)] \leq A[i]$. The root element $A[0]$ is the global minimum element.
2) Max-Heap Invariant: For every node i other than root, $A[\text{parent}(i)] \geq A[i]$. The root element $A[0]$ is the global maximum element.

3.2 Implicit Array Index Mapping

Because a Complete Binary Tree contains no internal structural gaps, it can be mapped directly into a contiguous zero-indexed array without allocating child pointers (left, right) or node wrappers.

For any element at index $i \in \{0, 1, \ldots, N-1\}$:

$$\text{leftChild}(i) = 2i + 1, \quad \text{rightChild}(i) = 2i + 2, \quad \text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor$$

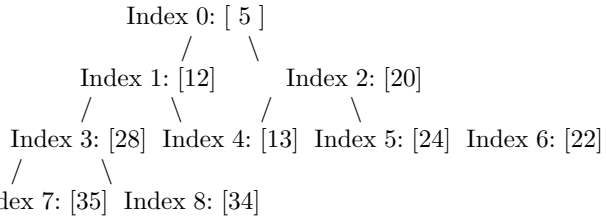


Fig. 1. Implicit contiguous array mapping of a Complete Binary Min-Heap.

4 Dynamic Operations: Insertion and Deletion

Binary heaps maintain structural and ordering invariants across insertion and deletion through two fundamental pointer-adjustment primitives: Up-Heapify and Down-Heapify.

4.1 Up-Heapify (Insertion / Push)

Inserting a new element into a min-heap proceeds in two stages:

1) Append: Add the element at index N (the tail of the array), preserving the Complete Binary Tree shape.
2) Percolate-Up (Up-Heapify): Compare the inserted element at index i with its parent $p = \lfloor (i-1)/2 \rfloor$. If $A[i] < A[p]$, swap $A[i]$ and $A[p]$ and set $i \leftarrow p$. Repeat until $A[i] \geq A[p]$ or $i = 0$.

Theorem 4 (Insertion Time Complexity). Inserting an element into a binary heap of size N takes $O(\log N)$ worst-case time complexity and $O(1)$ auxiliary space.

Proof. In the worst-case, the newly inserted key travels up from a leaf node to the root. The height of a complete binary tree with N nodes is $H = \lfloor \log_2 N \rfloor$. Each step performs constant $O(1)$ comparisons and swaps, yielding a total time bound of $O(H) = O(\log N)$. □

4.2 Down-Heapify (Extraction / Poll / Heapify)

Extracting the minimum element from a min-heap proceeds as follows:

1) Extract Root: Save the root element $A[0]$ to return.
2) Tail Replacement: Move the last element $A[N-1]$ to the root position $A[0]$ and decrement table size $N \leftarrow N - 1$.
3) Percolate-Down (Down-Heapify): Starting at index $i = 0$, compare $A[i]$ with its children $A[2i+1]$ and $A[2i+2]$. Identify the index m containing the smallest value among i, $2i+1$, and $2i+2$. If $m \neq i$, swap $A[i]$ with $A[m]$, set $i \leftarrow m$, and repeat.

Theorem 5 (Extraction Time Complexity). Extracting the root element from a binary heap of size N takes $O(\log N)$ worst-case time complexity and $O(1)$ auxiliary space.

5 In-Place $O(N)$ Build-Heap Construction (Floyd's Algorithm)

Given an arbitrary, unsorted array A of size N, we seek to transform A in-place into a valid binary heap.

5.1 Naive vs. Optimal Construction Strategies

- Strategy I (Sorting): Sort array A in $O(N \log N)$ time. While a sorted array satisfies heap invariants, sorting is unnecessary because heaps permit partial ordering.
- Strategy II (Repeated Up-Heapify Insertion): Insert elements $A[0 \dots N-1]$ one by one into an initially empty heap. The k-th insertion takes $O(\log k)$ time, yielding total time:

$$T(N) = \sum_{k=1}^{N} \log k = \log(N!) = \Theta(N \log N)$$

- Strategy III (Floyd's Bottom-Up Build-Heap): Process non-leaf nodes in reverse array order from index $\lfloor N/2 \rfloor - 1$ down to 0, calling downHeapify(A, N, i) on each node.

5.2 Floyd's Algorithm Implementation

```
void buildHeap(int[] arr) {
   int n = arr.length;
   // Index of last non-leaf parent node = (n/2) - 1
   for (int i = (n / 2) - 1; i >= 0; i--) {
      downHeapify(arr, n, i);
   }
}
```

6 Mathematical Proof of $O(N)$ Linear Time Build-Heap Complexity

We present a formal proof demonstrating that Floyd's bottom-up buildHeap runs in $O(N)$ linear time.

Theorem 6 (Linear Complexity of Floyd's Build-Heap Algorithm). Floyd's bottom-up heap construction algorithm transforms an unsorted array of size N into a binary heap in $O(N)$ time complexity.

Proof. Consider a Complete Binary Tree of height $h = \lfloor \log_2 N \rfloor$. Let level 0 denote the root, and level h denote the leaf level.

The maximum downward distance a node at level l can travel during downHeapify is $h - l$. The maximum number of nodes residing at level l in a complete binary tree of size N is bounded by:

$$n_l = \left\lceil \frac{N}{2^{l+1}} \right\rceil$$

Summing the total work $T(N)$ across all levels $l = 0, 1, \dots, h$:

$$T(N) = \sum_{l=0}^{h} n_l \cdot (h - l) = \sum_{l=0}^{h} \left\lceil \frac{N}{2^{l+1}} \right\rceil (h - l)$$

Letting $k = h - l$ represent height above leaf level, as l goes from 0 to h, k goes from h down to 0:

$$T(N) \leq \sum_{k=0}^{h} \frac{N}{2^{h-k+1}} \cdot k = \frac{N}{2^{h+1}} \sum_{k=0}^{h} k \cdot 2^k$$

Noting that $2^h \leq N < 2^{h+1}$, we simplify the fraction $\frac{N}{2^{h+1}}$ and express the summation in standard ratio form:

$$T(N) \leq \sum_{k=1}^{h} \frac{N}{2^{k+1}} \cdot k = \frac{N}{4} \sum_{k=1}^{h} \frac{k}{2^{k-1}}$$

To evaluate the infinite sum $S = \sum_{k=1}^{\infty} k x^{k-1}$ for $x = \frac{1}{2}$, consider the geometric series formula for $|x| < 1$:

$$g(x) = \sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$$

Differentiating both sides with respect to x:

$$g'(x) = \frac{d}{dx}\left(\sum_{k=0}^{\infty} x^k\right) = \sum_{k=1}^{\infty} k x^{k-1} = \frac{d}{dx}\left(\frac{1}{1-x}\right) = \frac{1}{(1-x)^2}$$

Evaluating $g'(x)$ at $x = \frac{1}{2}$:

$$S = g'\left(\frac{1}{2}\right) = \frac{1}{\left(1 - \frac{1}{2}\right)^2} = \frac{1}{\left(\frac{1}{2}\right)^2} = 4$$

Substituting $S = 4$ back into the work upper bound:

$$T(N) \leq \frac{N}{4} \cdot S = \frac{N}{4} \cdot 4 = N = O(N)$$

Thus, Floyd's bottom-up buildHeap construction achieves $O(N)$ linear time complexity. □

7 Complete Java Reference Implementation

Below is a complete, reference implementation of a generic MinHeap data structure in Java, featuring dynamic expansion, insertion, min extraction, and $O(N)$ array heapification.

```
import java.util.Arrays;
import java.util.NoSuchElementException;

public class CustomMinHeap {
    private int[] heap;
    private int size;
    private int capacity;

    public CustomMinHeap(int initialCapacity) {
        this.capacity = initialCapacity;
        this.size = 0;
        this.heap = new int[initialCapacity];
    }

    // O(N) Constructor using Floyd's buildHeap algorithm
    public CustomMinHeap(int[] array) {
        this.capacity = Math.max(array.length * 2, 10);
        this.size = array.length;
        this.heap = Arrays.copyOf(array, capacity);
        buildHeap();
    }

    private int parent(int i) { return (i - 1) / 2; }
    private int leftChild(int i) { return 2 * i + 1; }
    private int rightChild(int i) { return 2 * i + 2; }

    private void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    private void ensureCapacity() {
        if (size >= capacity) {
            capacity *= 2;
            heap = Arrays.copyOf(heap, capacity);
        }
    }

    public int peek() {
        if (size == 0) throw new NoSuchElementException("Heap is empty");
        return heap[0];
    }

    // O(log N) Up-Heapify Insertion
    public void insert(int val) {
        ensureCapacity();
        heap[size] = val;
        int i = size;
        size++;
        percolateUp(i);
    }

    private void percolateUp(int i) {
        while (i > 0 && heap[parent(i)] > heap[i]) {
            swap(i, parent(i));
            i = parent(i);
```

```
        }
    }

    // O(log N) Down-Heapify Extraction
    public int extractMin() {
        if (size == 0) throw new NoSuchElementException("Heap is empty");
        int minVal = heap[0];
        heap[0] = heap[size - 1];
        size--;
        if (size > 0) {
            percolateDown(0);
        }
        return minVal;
    }

    private void percolateDown(int i) {
        while (leftChild(i) < size) {
            int smallest = i;
            int l = leftChild(i);
            int r = rightChild(i);

            if (heap[l] < heap[smallest]) {
                smallest = l;
            }
            if (r < size && heap[r] < heap[smallest]) {
                smallest = r;
            }

            if (smallest == i) break;

            swap(i, smallest);
            i = smallest;
        }
    }

    // O(N) In-Place Build Heap
    private void buildHeap() {
        for (int i = (size / 2) - 1; i >= 0; i--) {
            percolateDown(i);
        }
    }

    public int size() { return size; }
    public boolean isEmpty() { return size == 0; }
}
```

8 Comparative Complexity Analysis & Systems Applications

Table 2 provides a comprehensive asymptotic performance breakdown of binary heaps compared to alternative priority data structures.

TABLE 2
Comprehensive Asymptotic Complexity Comparison of Priority Queue Structures

Data Structure	Find Min/Max	Extract Min/Max	Insert	Build Heap	Space Complexity
Unsorted Array	$O(N)$	$O(N)$	$O(1)$	$O(N)$	$O(N)$
Sorted Array	$O(1)$	$O(1)$	$O(N)$	$O(N \log N)$	$O(N)$
Balanced BST (AVL / Red-Black)	$O(\log N)$	$O(\log N)$	$O(\log N)$	$O(N \log N)$	$O(N)$
Binary Heap (Array CBT)	$O(1)$	$O(\log N)$	$O(\log N)$	$O(N)$	$O(N)$
Fibonacci Heap	$O(1)$	$O(\log N)$ amortized	$O(1)$	$O(N)$	$O(N)$

8.1 Real-World Systems Engineering Applications

1) Operating System Schedulers: Linux kernel Completely Fair Scheduler (CFS) and real-time task schedulers utilize priority queues to allocate CPU execution slices to ready processes based on static and dynamic priorities.
2) Graph Shortest Path Algorithms: Dijkstra's algorithm and Prim's Minimum Spanning Tree algorithm rely on min-heaps to select the next closest unvisited node in $O((V + E) \log V)$ time.
3) K-Way Stream Merging: High-throughput database query engines (such as PostgreSQL and Apache Spark) use min-heaps to perform external merge-sort across partitioned disk runs.
4) Top-K Frequency Processing: Stream processing engines (such as Apache Flink and Kafka Streams) maintain bounded min-heaps of size K to compute continuous Top-K trending elements over streaming windows in $O(N \log K)$ time.

9 Conclusion

This paper has presented a rigorous, unified formal analysis of binary heaps, priority queue paradigms, array-based complete binary tree indexing, and dynamic operations. We demonstrated that for dynamic datasets requiring frequent minimum/maximum access, binary heaps provide optimal $O(\log N)$ insertion and deletion bounds with zero pointer overhead.

We formally proved via Dirichlet exchange arguments that the Connect Ropes greedy choice property is optimal. Furthermore, using infinite arithmetic-geometric series differentiation, we provided a rigorous mathematical proof showing that Floyd's bottom-up buildHeap algorithm constructs a binary heap in $O(N)$ linear time. Finally, we presented a complete Java reference implementation and surveyed practical systems applications across kernel scheduling, graph optimization, and stream processing.